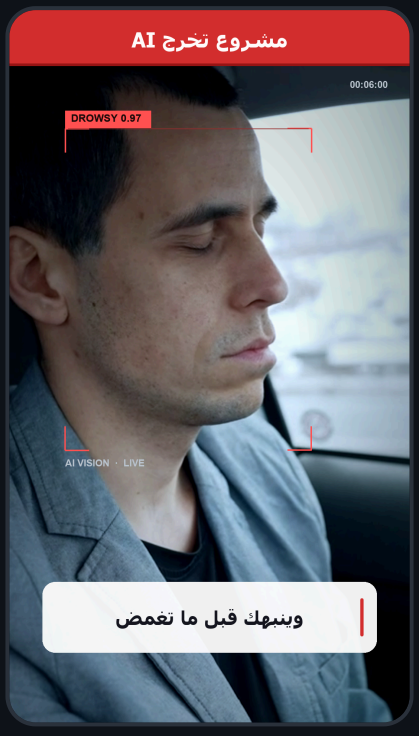


لقطة من الهوية البصرية للمشروع

DrowsyGuard AI

نظام ذكي يراقب السائق لحظة بلحظة،
ويكشف النعاس والمطبات وإشارة المرور الحمراء.

98.7% دقة كشف المطبات

99 اختبار ناجح

5 حالات للسائق

إعداد: محمود عبده

GitHub: MahmoudAbd0o / DrowsyGuard-AI-DEPI

المحتوى

01 المشروع في صفحة واحدة
02 إحنا وصلنا لفين؟
03 التقسيمة اللي عملناها
04 مجلد ai — الدماغ
05 مجلد vision — العين
06 الهاردوير والتسجيل والأدوات
07 سير البيانات
08 جدول الملخص
09 الأرقام والنتائج
10 تقييم جلسة القيادة
11 الناقص والخطة
12 قاموس المصطلحات
13 أسئلة متوقعة
14 سكريبت العرض

المشروع في صفحة واحدة

المشكلة، الحل، والمخرجات — من غير تفاصيل تقنية.

المشكلة

- **النعاس:** لحظة إغلاق العين لثانيتين كفاية لحادث.
- **المطبات:** غير معلومة وغير موحّدة الشكل.
- **الإشارة الحمراء:** السائق المشتّت يعدّيها بدون ما ياخد باله.

الحل

- كاميرا واحدة + ذكاء اصطناعي بيقرأ الوش والعين.
- موديل بيكشف المطبات على الطريق.
- عدّاد للإشارة الحمراء + تنبيه فوري (صوت + ليد + اهتزاز).

الفكرة في سطر واحد

نظام واحد بيجمع **رؤية حاسوبية** (قراءة السائق والطريق) مع **قرار فوري** و**تنبيه مادي** عن طريق الأردوينو — عشان ينبّه قبل ما الحادث يحصل، مش بعده.

مين بيعمل إيه؟

المكوّن	وظيفته	التقنية
مراقبة السائق	قياس فتح العين، التثاؤب، ميل الرأس، والالتفات	MediaPipe Face Mesh
قرار النعاس	تحويل القياسات لحالة واحدة واضحة كل لحظة	قواعد + PERCLOS
كشف المطبات	تحديد مكان المطب في الصورة قبل الوصول له	YOLOv8 (موديل مدرّب)
إشارة المرور	تمييز الأحمر/الأخضر + عدّاد مدة الوقوف	YOLOv8 + تحليل لون
التنبيه	ليد + بازر + اهتزاز حسب خطورة الحالة	Arduino + Serial
التسجيل والتقييم	حفظ كل لحظة في CSV وحساب الدقة	Python + Pandas

المخرج النهائي: تطبيق واحد (Dashboard) + موديل مدرّب + تقرير تقييم بالأرقام + فيديو تعريفي.

إحنا وصلنا لفين؟

حالة كل جزء بشكل صريح — إيه اللي خلص، وإيه اللي لسه.

المكوّن	الحالة	الدليل / الملاحظات
كشف النعاس (العين + الرأس)	مكتمل	شغال لايف على الكاميرا + داشبورد مستقر
PERCLOS بالوقت	مكتمل	نافذة 2 ثانية + نسبة تغطية + اختبارات
محرك القرار (5 حالات)	مكتمل	ALERT / LOW RISK / DISTRACTED / DROWSY / UNKNOWN
موديل المطبات YOLOv8	مكتمل	mAP@50 = 99.4% على 277 صورة تحقق
كشف إشارة المرور + العدّاد	مكتمل	دوال منفصلة + اختبارات بدون كاميرا
بروتوكول الأردوينو	مكتمل	5 أوامر + منع اهتزاز الإنذار (Debounce)
التسجيل والتقييم	مكتمل	CSV لكل فريم + Precision / Recall / Latency
تقرير التقييم	مكتمل	report.md + 3 رسوم بيانية
الفيديو التعريفي	عينة جاهزة	10 ثواني بأسلوب الريفيرنس، والفيديو الكامل لاحقًا
تجميع الهاردوير النهائي	قيد التنفيذ	الكود جاهز — التجربة المجمّعة مع الزميل
اختبار المطبات ميدانيًا	متبقي	محتاج ~40 صورة حقيقية من الطريق
حساسات MPU6050 / MQ-3	متبقي	قرار الشراء والتركيب

الخلاصة: كل السوفتوير مكتمل ومُختبَر (99 اختبار)، والناقص عمليّات تركيب وتجربة ميدانية.

نسبة الإنجاز الحالية



التقسيمة اللي عملناها

كل حاجة في مكانها: قرار واحد، رؤية واحدة، هاردوير واحد... عشان أي حد يلاقي حاجته بسرعة.

```
DrowsyGuard-AI-DEPI-main/
│
├── config.py            # كل الأرقام والعتبات في مكان واحد
├── main.py              # نقطة التشغيل (سطر واحد)
├── dashboard.py         # الواجهة الرسومية
│
├── ai/                  # الدماغ: القياسات والقرار
│   ├── features.py      # EAR / MAR / ميل الرأس
│   ├── perclos.py       # نسبة إغلاق العين بالوقت
│   ├── drowsiness.py    # تحويل القياسات لحالة
│   └── attention.py     # مؤشر انتباه 0-100
│
├── vision/              # العين: الكاميرا والكشف
│   ├── camera.py        # كاميرا + إعادة اتصال
│   ├── face_detector.py # MediaPipe + اختيار السواق
│   └── road_scene.py    # مطبات + إشارة مرور
│
├── hardware/            # الأوامر الخارجية
│   └── protocol.py      # 5 أوامر + Debounce
├── arduino_bridge/
│   └── arduino_bridge.ino # كود الأردوينو
│
├── data/                # التسجيل والتقييم
│   ├── logger.py        # CSV + سجل الأحداث
│   └── evaluate.py      # Precision / Recall / Latency
│
├── tests/               # 9 اختبار — 99 ملفات
├── tools/               # محاكاة / تقرير / فيديو / داتاسيت
├── notebooks/           # تدريب الموديل على Kaggle
├── report/              # تقرير التقييم + الرسوم
└── demo_video/          # الفيديو التعريفي
```

🔴 ليه التقسيمة دي؟

المبدأ	الفايدة العملية
كل حاجة في مكانها	ملف واحد بيعمل حاجة واحدة، فالتعديل مبيخربش حاجة تانية
فصل الرؤية عن القرار	نقدر نختبر القرار بدون كاميرا، ونجرّب موديل جديد لوحده
أرقام في ملف واحد	تغيير حساسية النظام = تعديل رقم في config.py بس
اختبارات مستقلة	أي تعديل بيتم التأكد منه فورًا قبل التجربة على الجهاز

1. مجلد ai/ — الدماغ

بياخد نقاط الوش ويحوّلها لأرقام... والأرقام تتحوّل لقرار.

`ai/features.py`

قياس المؤشرات من نقاط الوش

بياخد 468 نقطة على الوش ويحسب منها: **EAR** (نسبة فتح العين)، **MAR** (فتحة البق = تثاؤب)، **ميل الرأس** (النفس بينزل تحت خط العين)، و**الالتفات** (بُعد الأنف عن منتصف الوش). لو القياس غلط أو الوش مش واضح بيرجّع **None** — مش صفر — عشان النظام ميفتكرش إن العين مفتوحة.

`ai/perclos.py`

PERCLOS — أهم مقياس في المشروع

PERCLOS = نسبة الوقت اللي كانت العين مقفولة فيه خلال آخر **2 ثانية**. الحساب **بالوقت الحقيقي** مش بعدد الفريمات، فلو الكاميرا بطّلت النتيجة تفضل صح. وبيقول نسبة التغطية (Coverage): لو القياسات قليلة، الحالة تبقى **UNKNOWN** مش إنذار كاذب.

`ai/drowsiness.py`

محرك القرار — قواعد واضحة ومقصودة

DROWSY لو: PERCLOS ≥ 40% — أو الرأس نازل مع إغلاق عين — أو تثاؤبين في 90 ثانية.
DISTRACTED لو الرأس ملفوف بعيد عن الطريق لمدة كافية.
LOW RISK لو PERCLOS بين 15% و40% أو تثاؤب حالي.
ALERT لو العين مفتوحة والرأس مستقيم. **UNKNOWN** لو الوش مش ظاهر.

`ai/attention.py`

مؤشر الانتباه 0–100

رقم واحد بسيط يعرض للسائق: بيبدأ من 100 وينقص مع إغلاق العين، والرأس النازل (−10)، والالتفات (−15)، والتثاؤب (−8). وفيه سقوف: النعاس ≥ 35، الالتفات ≥ 55.

قرار تصميمي مهم: «الوش مش ظاهر» أو «القياس غلط» = UNKNOWN (بدون إنذار صوتي)، مش ALERT. ده بيقلل الإنذارات الكاذبة اللي بتخلي السواق يفصل النظام.

2. مجلد vision/ — العين

الكاميرا، الوش، والطريق. كله معزول عن باقي النظام عشان يتختبر لوحده.

`vision/camera.py`

الكاميرا + إعادة الاتصال التلقائي

بيفتح أول كاميرا شغالة، بيسحّنها (قراءة كام فريم)، ولو الكاميرا قطعت بيحاول يرجّعها لوحده بدل ما البرنامج يقع.
الـDashboard بيطلب «فريم» وهو مش مسؤول عن أي تفاصيل تقنية.

`vision/face_detector.py`

كشف الوش + اختيار السواق

بيستخدم **MediaPipe Face Mesh** ويرجّع نقاط الوش كأرقام عادية (Numpy) عشان باقي الكود ما يعتمدش على مكتبة معينة. ولو فيه أكتر من وش، بيختار **السواق**: أكبر وش + أقرب لمكان السواق (في مصر: ناحية اليمين في وضع المراية).

`vision/road_scene.py`

الطريق: مطبات + إشارة مرور

- **TrafficLightDetector:** موديل YOLO جاهز بيعرف إشارات المرور، ولونه بيتحدد بدالة light_state() (أحمر/أصفر/أخضر/غير معروف) من لون الجزء المضيء.
- **BumpDetector:** موديل bump_model.pt مدرّب عندنا على فئة واحدة: مطب.
- **RedLightTimer:** بيحسب ثواني الاستمرار على الأحمر، ويصفّر العدّاد أول ما الإشارة تتغيّر.

● ليه المطبات محتاجة موديل خاص؟

لأن المطب مش «شيء» ثابت الشكل زي العربية أو الشخص. الموديل العام (COCO) مبيعرفوش خالص. عشان كده عملنا داتاسيت من صور طرق حقيقية، ودرّبنا موديل مخصوص، وحوّلناه لفئة واحدة (0 = مطب) عشان يبقى سريع ودقيق.

المرحلة	الأداة	المخرج
تجهيز الداتاسيت	tools/prepare_bump_dataset.py	صور + ليبلز + data.yaml
التدريب (GPU)	Kaggle + notebook	bump_model.pt
الاستخدام	yolo_phase2.py	بوكس على المطب في الفيديو

3. الهاردوير والتسجيل والأدوات

إزاي القرار بيوصل للسواق، وإزاي بنحفظ ونقيّم كل لحظة.

hardware/protocol.py — لغة التفاهم مع الأردوينو

الأمر	الحالة	رد فعل الأردوينو
0	آمن	ليد أخضر — بدون صوت — الموتور شغال
1	خطر منخفض	ليد أصفر + بيب خفيف + اهتزاز 30%
2	نعاس (DROWSY)	ليد أحمر + إنذار عالي + اهتزاز 100% + إيقاف الموتور
3	التفات (DISTRACTED)	ليد أصفر وامض + بيب مزدوج + اهتزاز 60%
N	وش غير ظاهر	ليد أصفر وامض ببطء — بدون صوت

LevelDebouncer: الحالة لازم تفضل ثابتة 5 فريمات قبل ما تتبعت، عشان الإنذار ميرفرفش بين حالة وحالة في لمح البصر.

data/ — تسجيل وتقييم

`data/logger.py`

بيسجّل كل فريم (القياسات + الحالة) في CSV بشكل مخزّن مؤقتًا، فالكتابة على القرص مبطّأش الفيديو. وفيه سجل أحداث مقروء للإنذارات.

`data/evaluate.py`

يحسب Precision / Recall / F1 / نسبة الإنذارات الكاذبة / التغطية / زمن الاكتشاف — على أي ملف مُعلَّم.

tools/ — الأدوات اللي بتولّد المخرجات

الأداة	بتعمل إيه
simulate_session.py	جلسة 95 ثانية مكتوبة، بتستخدم نفس المحرك الحقيقي عشان التقرير يبقى قابل للتكرار
make_report.py	يبني تقرير التقييم + الرسوم البيانية من نتائج الموديل والجلسة
prepare_bump_dataset.py	يفلتر داتاسيت ويحوّله لفئة واحدة (مطب) ويقسّمه تدريب/تحقق
make_video.py	الفيديو التعريفي 9:16 بأسلوب الريفيرنس

سير البيانات من الكاميرا للتنبيه

إزاي الصورة بتتحوّل لحالة... والحالة بتتحوّل لإنذار.

المسار الأول: مراقبة السائق

1. **كاميرا** — 640×480، مع إعادة اتصال تلقائي
2. **MediaPipe** — 468 نقطة على الوش (أرقام إحداثيات)
3. **ai/features** — EAR (العين) + MAR (التثاؤب) + ميل الرأس + الالتفات
4. **ai/perclos** — نسبة إغلاق العين خلال 2 ثانية + نسبة تغطية القياس
5. **ai/drowsiness** — القرار: ALERT / LOW RISK / DISTRACTED / DROWSY / UNKNOWN
6. **hardware/protocol** — أمر واحد + Debounce (5 فريمات)
7. **Arduino** — ليد + بازر + اهتزاز، والموتور بيقف في حالة النعاس

المسار الثاني: الطريق

1. **كاميرا** — نفس الفريم
2. **YOLOv8** — موديل جاهز للإشارات + موديلنا الخاص للمطبات
3. **تنبيه** — بوكس على المطب، وحالة الإشارة + عدّاد الأحمر

ملاحظة صريحة للمناقشة: المسارين شغالين ومُختبَرين كل واحد لوحده. دمجهم في واجهة واحدة (شاشة تعرض السواق والطريق مع بعض) هو خطوة المرحلة الجاية.

جدول الملخص

لو حافظت الجدول ده، تبقى فاهم كل حاجة في المشروع.

الجزء	وظيفته	بيشتغل إزاي	الملف
الإعدادات	كل الأرقام في مكان واحد	عتبات العين والنعاس والالتفات والكاميرا والسيريال	`config.py`
الواجهة	شاشة المشروع	كاميرا لايف + الحالة + EAR/MAR + إنذارات + تسجيل	`dashboard.py`
قياسات الوش	تحويل النقاط لأرقام	EAR للعين، MAR للتثاؤب، نسب ميل الرأس والالتفات	`ai/features.py`
PERCLOS	نسبة إغلاق العين	نافذة زمنية 2 ثانية + تجاهل القياسات الغلط	`ai/perclos.py`
القرار	تحديد حالة السائق	قواعد ثابتة على PERCLOS + الرأس + التثاؤب	`ai/drowsiness.py`
الانتباه	رقم 0–100 للسائق	100 ناقص خصومات حسب كل مؤشر	`ai/attention.py`
الكاميرا	تغذية الصورة	فتح + تسخين + إعادة اتصال عند القطع	`vision/camera.py`
كشف الوش	تحديد السواق	MediaPipe + اختيار أكبر وش قريب من مكان السواق	`vision/face_detector.py`
الطريق	مطبات وإشارة مرور	YOLO للمطبات + YOLO للإشارات + تحليل اللون والعدّاد	`vision/road_scene.py`
الأوامر	التحكم في الهاردوير	5 أوامر (N/0/1/2/3) + تثبيت الحالة 5 فريمات	`hardware/protocol.py`
الأردوينو	التنبيه المادي	ليد + بازر + اهتزاز + إيقاف الموتور عند النعاس	`arduino_bridge.ino`
التسجيل	حفظ الجلسة	CSV لكل فريم (مخزّن) + سجل أحداث	`data/logger.py`
التقييم	قياس دقة النظام	Precision / Recall / F1 / إنذارات كاذبة / زمن اكتشاف	`data/evaluate.py`
الأدوات	توليد المخرجات	محاكاة جلسة + تقرير + فيديو + داتاسيت	`tools/`
الاختبارات	حماية الكود	99 اختبار على القرار والقياسات والبروتوكول والتقرير	`tests/`

إزاي تستخدم الجدول في المناقشة: ابدأ بالجزء، قول وظيفته، بعدين جملة واحدة عن طريقة عمله، واقفل باسم الملف. كده كل إجابة تبقى 15 ثانية ومنظمة.

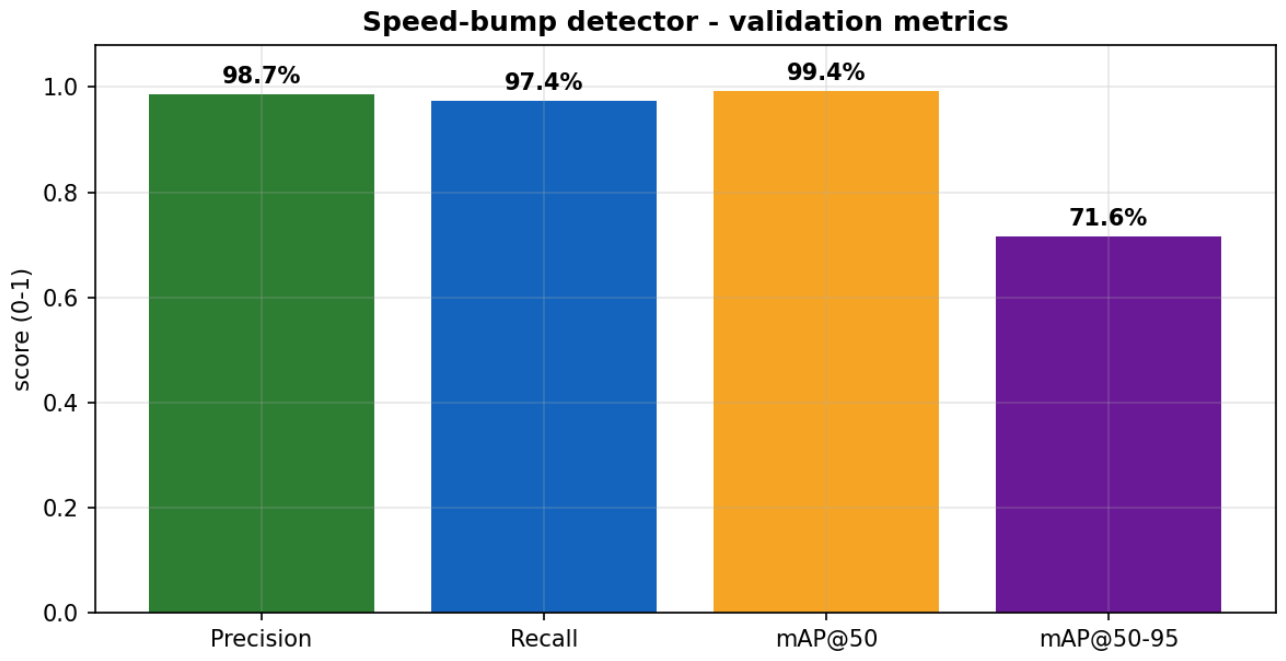
الأرقام والنتائج

كل رقم هنا له مصدر قابل لإعادة التشغيل.

98.7%	97.4%	99.4%	71.6%
Precision	Recall	mAP@50	mAP@50-95

موديل كشف المطبات

البند	القيمة
الموديل الأساسي	YOLOv8n — صورة 640 — Epoch 50
بيانات التحقق	277 صورة / 305 حالة مطب
الفئة	فئة واحدة: speedbump
حجم الموديل	6.35 MB (سريع على CPU)



منحنيات التدريب — التحقق من عدم الـOverfitting

ليه mAP@50-95 أقل؟

لأنها بتقيس دقة **حدود البوكس** عند كل نسب التداخل من 50% لـ95%، وده أصعب بكتير من مجرد «هل كشف المطب ولا لأ». الرقم 71.6% كويس جدًا لموديل صغير على فئة واحدة.

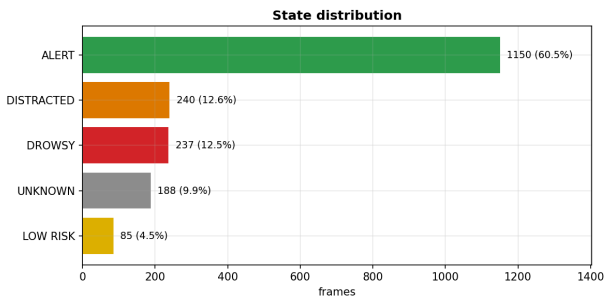
تقييم جلسة القيادة

جلسة 95 ثانية بسيناريو ثابت — القياسات والقرار من محرك المشروع الحقيقي.

95s	1900	91.6%
مدة الجلسة	فريم	تغطية الوش



خط زمني: PERCLOS والحالة على مدار الجلسة



توزيع الحالات على الجلسة

الحالة	النسبة
ALERT	60.5%
DISTRACTED	12.6%
DROWSY	12.5%
UNKNOWN	9.9%
LOW RISK	4.5%

المؤشر	القيمة	معناه
متوسط PERCLOS	13.8%	السواق طبيعي في أغلب الوقت
أقصى PERCLOS	73.2%	لحظة نعاس حقيقية اتكشفت
متوسط الانتباه	78 / 100	مستوى جيد مع فترات انخفاض

كن صريحًا: دي **محاكاة مضبوطة** (السيناريو مكتوب)، لكن القياسات والقرار ناتجين من نفس الكود اللي بيشتغل لايف — وده اللي يخليها دليل على صحة المنطق، مش على الأداء في الشارع.

اللي ناقص والخطة

الاعتراف بالحدود + خطة واضحة = ثقة أعلى من ادعاء الكمال.

الحدود الحالية

النقطة	التفصيل	الأثر
اختبار المطبات ميدانيًا	الموديل متدرّب ومقيّم، لكن بدون صور من طريقنا	ثقة أقل في الواقع
دمج الهاردوير	البروتوكول جاهز — التجميع الفعلي مع الزميل	العرض المادي لسه
حساسات إضافية	MPU6050 (اهتزاز/ميل) و MQ-3 (كحول)	لم تُضَف بعد
ظروف صعبة	ليل / شمس مباشرة / نضارة شمسية	دقة أقل
مسارين منفصلين	النعاس في الداشبورد، والطريق في سكريبت مستقل	محتاج شاشة واحدة

خطة المرحلة الجاية (قبل التسليم النهائي)

الأسبوع	الشغل	المخرج
الأول	تصوير ~40 صورة مطبات من الطريق + تقييم الموديل عليها	رقم دقة واقعي
الثاني	تجميع الأردوينو + تجربة الأمر N/0/1/2/3 على الجهاز	عرض مادي شغال
الثالث	شاشة واحدة تجمع النعاس + المطبات + الإشارة	نسخة نهائية
الرابع	الفيديو الكامل 45–60 ثانية + بروفة العرض	تسليم جاهز

جملة تقولها للمناقش: «النظام الأساسي مكتمل ومُختبَر بـ99 اختبار، والناقص هو التحقق الميداني والتجميع النهائي — وده مخطط له في ٤ أسابيع.»

قاموس المصطلحات

لغة بسيطة لأي مصطلح ممكن يتسأل عنه.

المصطلح	يعني إيه بالمصري
AI (ذكاء اصطناعي)	برنامج بيتعلم من أمثلة بدل ما نكتبله كل الاحتمالات بإيدينا
MediaPipe Face Mesh	مكتبة بترسم 468 نقطة على الوش من صورة الكاميرا
EAR	نسبة فتح العين: رقم بيصغر لما العين تقفل
MAR	نسبة فتح البق: بيكبر لما تتثاوب
PERCLOS	نسبة الوقت اللي كانت العين مقفولة فيه خلال فترة — المعيار العلمي للنعاس
YOLO	موديل بيكشف الأجسام في الصورة ويحدد مكانها بسرعة (لحظيًا)
Precision	لما النظام يقول «فيه مطب»، بيبقى صح قد إيه؟ (98.7%)
Recall	من كل المطبات الحقيقية، كشف كام واحدة؟ (97.4%)
mAP@50	مقياس دقة الكشف عند تداخل 50% بين البوكس والحقيقة
Debounce	تثبيت الحالة كام فريم قبل التنفيذ عشان الإنذار ميرفرفش
Serial	خط اتصال بسيط بين الكمبيوتر والأردوينو
UNKNOWN	حالة «مش قادر أقيس» — مش إنذار ومش أمان

أسئلة متوقعة وإجاباتها

كل إجابة في سطرين — قولها بثقة وبدون حفظ حرفي.

1) المشروع ده بيضيف إيه عن الأنظمة الجاهزة؟
بيجمع ٣ حاجات في نظام واحد: نعاس السواق + مطبات + إشارة مرور، بموديل مدرَّب عندنا، وكل جزء قابل للتعديل من مكان واحد.

2) ليه استخدمتوا PERCLOS مش مجرد «العين مقفولة»؟
لأن الرمشة العادية نص ثانية. PERCLOS بيقيس نسبة الإغلاق على مدى فترة، فبيفرّق بين رمشة طبيعية ونعاس حقيقي.

3) إيه اللي يمنع الإنذارات الكاذبة؟
تثبيت الحالة 5 فريمات، تجاهل القياسات غير الصالحة، وأي وش غير واضح = UNKNOWN بدون صوت.

4) دقة الموديل مبنية على إيه؟
277 صورة تحقق / 305 حالة مطب، والنتيجة Precision 98.7% وRecall 97.4% وmAP@50 99.4%. والتقرير فيه الرسوم والخطوات لإعادة التشغيل.

5) بيشتغل في الليل أو بالإضاءة الضعيفة؟
دي نقطة محدودة حاليًا. الحل: تحسين الإضاءة أو كاميرا IR، وإضافة ظروف الليل للاختبار في المرحلة الجاية.

6) إيه أصعب حاجة واجهتكم؟
إن السوفتوير يفضل ثابت: عالجنا مشاكل زي عدّاد PERCLOS بالفريمات، ووميض الإنذار، وفصل الكود لموديولات قابلة للاختبار.

7) المشروع قابل للتوسع؟
أيوه — كل حساس أو موديل جديد ليه مكان واضح: هاردوير في /hardware، ورؤية في /vision، وقرار في /ai.

8) إيه أضعف نقطة في المشروع؟
التحقق الميداني. الموديل قوي على بيانات التحقق، لكن لسه محتاج صور من طرق حقيقية لتأكيد الأداء.

> **قاعدة ذهبية في المناقشة:** لو اتسألت عن حاجة مش منفّذة، متخمّنش. قول: «دي جزء مخطط له ولسه، والسبب كذا، والخطوة الجاية كذا». الصدق الواضح بيكسب ثقة أكتر من إجابة كاملة غلط.

سكريبت العرض — 3 دقايق

قول الكلام ده بالترتيب وأنت بتفتح البرنامج.

0:00 – 0:25 · المقدمة

«المشروع اسمه DrowsyGuard AI. بيعالج ٣ أسباب رئيسية للحوادث: نعاس السواق، المطبات، وتخطي الإشارة الحمراء. الهدف إن النظام ينبّه قبل الحادث، مش يسجّله.»

0:25 – 1:10 · كشف النعاس

«الكاميرا بتقرأ 468 نقطة على الوش. منها بنحسب فتح العين EAR، والتثاؤب MAR، وميل الرأس. وبعدين نحسب PERCLOS: نسبة الوقت اللي كانت العين مقفولة فيه في آخر ثانيتين. لو زادت عن 40%، الحالة تبقى DROWSY، والأردوينو بيشغّل إنذار ويهدّي العربية.»

1:10 – 1:45 · المطبات وإشارة المرور

«موديل YOLOv8 مدرَّب عندنا على فئة واحدة: مطب — بدقة 98.7% و99.4% mAP@50. وبنفس الفكرة بنكشف إشارة المرور، وفيه عدّاد بيحسب مدة الاستمرار على الأحمر.»

1:45 – 2:20 · الدليل والأرقام

«فيه 99 اختبار آلي، وتقرير تقييم فيه منحنيات التدريب وخط زمني لجلسة 95 ثانية، وكل الأرقام قابلة لإعادة التشغيل بأمر واحد.»

2:20 – 2:45 · الحدود والخطة

«الناقص هو التحقق الميداني وتجميع الهاردوير، ومخطط له في ٤ أسابيع: تصوير صور مطبات حقيقية، تجميع الأردوينو، شاشة واحدة، والفيديو النهائي.»

2:45 – 3:00 · الختام

«الفكرة مش جهاز جديد، لكن طبقة أمان ذكية تقدر تتضاف لأي عربية بكاميرا.»

الكود الكامل على GitHub

github.com/MahmoudAbd0o/DrowsyGuard-AI-DEPI

امسح الكود من الموبايل في أي وقت قدام المناقش.